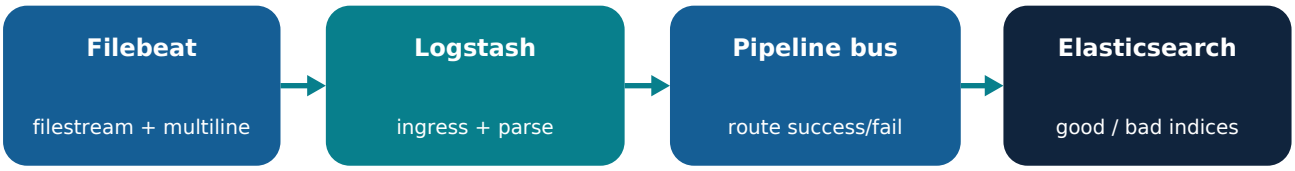
ELASTIC STACK RELIABLE INGESTION & DATA LIFECYCLE LABS

Filebeat | Logstash | Elasticsearch | Kibana



L3: PQ buffers output interruptions | DLQ handles eligible indexing rejects

LAB	DELIVERABLE	TIME
SETUP	Install and verify Filebeat and Logstash on Ubuntu	45-70 min
L3 | Reliable ingestion	Multiline, Grok/Date/Mutate, routing, queues, load test	150 min
L4 | Data management	Ingest simulation, mapping, templates, data streams, ILM rollover	110 min
Q2 | Query DSL	Match, term, range, bool filter with validated results	45 min
Extension	JSON/CSV/XML parsers, DLQ and licensed central management	45-60 min

Environment: Ubuntu lab virtual machine(s); self-managed Elastic Stack 8.19+ / 9.x; TLS-enabled Elasticsearch; Kibana Dev Tools; Filebeat and Logstash.

Prepared: 02 November 2026 | Audience: operations engineers, platform engineers and technical trainers

Includes a separate ZIP with complete configuration files, JSON request bodies, sample data and a bounded load generator.

FIELD-UPDATED EDITION | Verified four-VM setup and 9.5.5 troubleshooting

1 | Lab scope, learning outcomes and boundaries

This manual continues from the uploaded Day 1 Ubuntu/KVM lab (L1, L2 and Q1). It adds explicit Logstash and Filebeat installation before L3. Elasticsearch and Kibana must already work on the Day 1 VMs. All index names, pipeline IDs and paths are prefixed for the lab. Run experiments only against a dedicated lab or disposable cluster; do not stop or reconfigure a shared production cluster.

EXERCISE	YOU WILL PROVE
L3.1	Select app log files; exclude debug files; consolidate stack traces as one event.
L3.2	Extract fields with Grok; normalize with Mutate and Date; send malformed events to quarantine.
L3.3	Connect four Logstash pipelines; distinguish PQ buffering, parser quarantine and DLQ.
L3.4	Create an isolated output interruption; inspect queue buildup; measure EPS and bottleneck.
L4.1	Simulate an ingest pipeline and verify each processor before indexing.
L4.2	Apply component/index templates, inspect explicit and newly inferred mapping fields.
L4.3	Index a data stream; roll it over; inspect ILM and backing indices.
Q2	Validate match, term, range and bool.filter with expected document IDs.

Prerequisite checklist

- Elasticsearch and Kibana versions are compatible; Logstash and Filebeat versions are on their supported matrix.
- You have sudo on the Logstash/Filebeat Ubuntu VM and a permitted test account with privileges for indexing, ingest pipelines, templates, data streams and ILM.
- Dedicated test Logstash pipeline ports 5044 (Beats input) and 9600 (local monitoring API) are not already in use.
- Allow at least 3 GB free disk for lab files and PQs; 4 GB RAM for a small Logstash training VM is a practical starting point.
- Security is enabled: use a valid Elasticsearch CA cert, HTTPS host, and credentials stored in the Logstash keystore.
- Snapshots are available before making lifecycle or security changes. The example ILM policy can delete lab data after one day.

TRAINER NOTE The companion Filebeat configuration shows example Logstash IP 192.168.122.21, which MUST be replaced with the actual VM address. Example cluster addresses are illustrations from Day 1, not actual machine IPs.

Trainer timing and pass criteria

TRACK	CHECKPOINT	PASS
L3	1 multiline stack trace + 1 malformed event	Good index + quarantine index
L3	Queue increases during interruption, drains after recovery	Metrics / screenshot and EPS report
L4	Six records, explicit + dynamic mapped fields	Template simulation + 2 backing indices
Q2	Expected result IDs for 4 queries	Saved Dev Tools responses

2A | Verified Day 2 deployment - actual 4-VM classroom setup

This section replaces the earlier Day 2 setup pages that recommended a fifth Logstash VM. It incorporates the terminal screenshots and Kibana results shared on 02 November 2026. Do not provision a new Logstash VM for the current exercise.

Actual machine / cluster member	Observed state	Training role
Ubuntu hostname es-node1	Kibana active; Elasticsearch inactive	Kibana + Logstash + Filebeat on this machine
Elasticsearch node01 - 192.168.122.227	Listed in Kibana _cat/nodes	Elasticsearch data / cluster member
Elasticsearch node02 - 192.168.122.246	Elected master (*) in screenshot	Elasticsearch data / cluster member
Elasticsearch node03 - 192.168.122.102	Listed in Kibana _cat/nodes	Elasticsearch data / cluster member

FIELD NOTE The OS hostname es-node1 is misleading: the shown services prove that this machine runs Kibana, while its local Elasticsearch service is inactive. The three Elasticsearch nodes are node01 / node02 / node03 in Kibana; do not rename or restart them to match earlier examples.

Observed installation and memory (not an assertion of completed ingestion)

Check	Evidence
Filebeat	Version 9.5.5 (amd64), service enabled, previously inactive
Logstash	Version 9.5.5 visible in startup logs; installed and enabled, previously inactive
/etc/logstash/conf.d/	Initially empty (total 0); later filebeat.conf was created
Kibana	active on es-node1
RAM on es-node1	31 GiB total; 27 GiB used; 3.1 GiB available; no swap

An active (running) Logstash or Filebeat service and a successful end-to-end Elasticsearch write have not yet been shown in the supplied screenshots. Complete the verification gates below before marking L3 as passed.

2A.1 | Check services and confirm the local layout

Terminal: run on es-node1

```
hostname
free -h
systemctl is-active elasticsearch
systemctl is-active kibana
systemctl is-active logstash
systemctl is-active filebeat
filebeat version
/usr/share/logstash/bin/logstash --version
sudo ls -l /etc/logstash/conf.d/
```

Expected according to the screenshots: hostname es-node1, Kibana active, Elasticsearch inactive, Logstash and Filebeat initially inactive. The final service state can change after you start the services.

Browser: verify the remote Elasticsearch cluster in Kibana Dev Tools

```
GET _cat/nodes?v
GET _cluster/health?pretty
GET _cat/nodes?v&h=name,ip,heap.percent,ram.percent,cpu
```

The screenshot showed node01 (192.168.122.227), node02 (192.168.122.246; elected master) and node03 (192.168.122.102). The cluster health color was not shown; record it now.

> FIELD NOTE The service state inactive for Elasticsearch on the Kibana machine is not automatically a cluster failure. Verify cluster health using Kibana before changing any Elasticsearch systemd service.

Resource gate

```
free -h
df -h /var/lib/logstash /var/log/logstash 2>/dev/null || true
ps -eo pid,comm,rss --sort=-rss | head -15
```

Available memory was only 3.1 GiB when measured, with swap disabled. A small parser demo may be feasible; avoid persistent-queue stress or high-throughput tests until memory headroom is sufficient. Do not restart the three remote Elasticsearch nodes to create an outage.

Stage the companion lab files on this machine

```
# Transfer the updated ZIP to the home directory of es-node1.
cd ~
sudo apt install -y unzip
unzip -o Elastic_Stack_Day2_Actual_Setup_Lab_Files.zip
cd ~/ELK_Lab_Files
ls sample-data filebeat logstash elasticsearch scripts
```

Use the newly provided Actual_Setup ZIP, not the earlier pack with a fifth-VM address. Keep the terminal in ~/ELK_Lab_Files when running relative file-copy commands.

2A.2 | Packages and the first Logstash pipeline

Filebeat and Logstash are already installed. Reinstallation is unnecessary. These commands are included for learners repeating the same setup on a clean Ubuntu training machine.

```
# Only if a package is missing, and the Elastic 9.x APT repo exists:
sudo apt update
sudo apt install -y filebeat logstash
sudo systemctl enable filebeat
sudo systemctl enable logstash
# "enable" does not start a previously stopped service.
```

Why the empty conf.d directory mattered

The command sudo ls -l /etc/logstash/conf.d/ returned total 0. Logstash had no starter pipeline defined in the default conf.d location, so the first lab pipeline was created here:

```
sudo nano /etc/logstash/conf.d/filebeat.conf
```

Paste this exact starter configuration

```
input {
  beats {
    host => "127.0.0.1"
    port => 5044
  }
}
filter {
  if [message] {
    mutate { add_field => { "lab_source" => "filebeat" } }
  }
}
output {
  stdout { codec => rubydebug }
}
```

Save in nano using Ctrl+O, Enter, Ctrl+X. This first stage sends events to standard output / Logstash logs, not yet Elasticsearch. Binding the Beats input to loopback is appropriate because both services run on the same machine.

> FIELD NOTE Keep the starter file until the initial end-to-end Filebeat-to-Logstash receiver test succeeds. When switching to the four-pipeline L3 setup later, remove/disable this starter pipeline to prevent a duplicate listener on TCP 5044.

2A.3 | Logstash refused root execution - correct validation

Observed command and message

```
# This was attempted; do NOT use it for validation:
sudo /usr/share/logstash/bin/logstash \
  --path.settings /etc/logstash --config.test_and_exit
# [FATAL] Logstash cannot be run as superuser.
```

The failure was caused by launching Logstash as the root account. It does not establish a syntax error in filebeat.conf. Logstash 9.x blocks superuser execution by default. Do not enable superuser execution as a workaround.

Correct command: run under the logstash service account

```
sudo -u logstash /usr/share/logstash/bin/logstash \
  --path.settings /etc/logstash --config.test_and_exit
```

Your subsequent screenshot showed Configuration OK and Config Validation Result: OK. This confirms syntactic validation of the loaded pipeline configuration, not a complete event delivery test. Grok correctness and service health must be tested separately.

Why other error lines appeared even after Configuration OK

The same test showed Log4j messages such as Unable to locate appender plain_rolling and a file access error for /var/log/logstash/logstash-plain.log (Permission denied). The log-file permission error must be resolved before treating the installation as healthy.

> FIELD NOTE The screenshot also showed Logstash creating queue and dead-letter-queue directories. Directory creation during startup is normal; it is not proof that a persistent queue or dead-letter queue has successfully processed events.

2A.4 | Resolve /var/log/logstash Permission denied

Inspect permissions before making a change

```
sudo ls -ld /var/log/logstash
sudo ls -l /var/log/logstash
sudo stat -c '%U:%G %a %n' /var/log/logstash \
  /var/log/logstash/logstash-plain.log 2>/dev/null || true
```

If a previous root-run attempt created the log directory or files with the wrong owner, restore ownership only for the Logstash log directory:

```
sudo chown -R logstash:logstash /var/log/logstash
sudo -u logstash test -w /var/log/logstash \
  && echo "Logstash log directory is writable"
```

Repeat the non-root configuration test

```
sudo -u logstash /usr/share/logstash/bin/logstash \
  --path.settings /etc/logstash --config.test_and_exit
```

Expected: Configuration OK without new permission-denied errors. If denied errors persist, inspect parent directories and the log4j2 configuration; do not chmod all Logstash directories to 777.

Start Logstash using systemd

```
sudo systemctl enable --now logstash
sudo systemctl status logstash --no-pager
sudo ss -lntp | grep ':5044'
sudo journalctl -u logstash -n 60 --no-pager
```

Pass gate: active (running) and a listener at 127.0.0.1:5044 (or 0.0.0.0:5044 if the starter config is intentionally bound to all interfaces). If not active, inspect journalctl before starting Filebeat.

2A.5 | Correct Filebeat output: localhost 9200 -> 5044

Your screenshot showed the default Filebeat section output.elasticsearch with hosts: ["localhost:9200"]. That is unsuitable for the planned Filebeat-to-Logstash workflow. Changing only the port while keeping output.elasticsearch is also incorrect: the output protocol must change.

Backup and edit /etc/filebeat/filebeat.yml

```
sudo cp /etc/filebeat/filebeat.yml \
  /etc/filebeat/filebeat.yml.before-logstash
sudo nano /etc/filebeat/filebeat.yml
```

Disable the Elasticsearch output and use Logstash output

```
# Comment out the existing Elasticsearch output and its settings:
#output.elasticsearch:
#  hosts: ["localhost:9200"]
#  preset: balanced
#  protocol: "https"

# Enable one output only:
output.logstash:
  hosts: ["127.0.0.1:5044"]
```

FIELD NOTE Do not leave another active output.elasticsearch option elsewhere in the YAML. Keep the existing Filebeat input or use the complete filestream input on the next page. localhost / 127.0.0.1 is correct only because Filebeat and Logstash share this machine.

Verify the configuration before starting

```
sudo filebeat test config -c /etc/filebeat/filebeat.yml -e
sudo filebeat test output -c /etc/filebeat/filebeat.yml -e
```

Pass gate: configuration test reports Config OK; output test establishes Beats protocol connectivity to the running Logstash receiver. A running service alone does not prove delivery.

2A.6 | Configure the log source and verify one event

Recommended L3 filestream input (keep only one filebeat.inputs key)

```
filebeat.inputs:
  - type: filestream
    id: training_app_logs
    enabled: true
    paths:
      - /var/tmp/elk-labs/logs/app-*.log
    exclude_files: ['app-debug\.log$']
    prospector.scanner.fingerprint.length: 64
    parsers:
      - multiline:
          type: pattern
          pattern: '^[0-9]{4}-[0-9]{2}-[0-9]{2}T'
          negate: true
          match: after
          timeout: 2s
          max_lines: 100
output.logstash:
  hosts: ["127.0.0.1:5044"]
```

For this lab, use the full filebeat-lab.yml included in the updated ZIP (it contains scanner interval and training fields). The 64-byte fingerprint setting supports tiny synthetic files, but lowers identity collision resistance; avoid changing fingerprint settings after Filebeat has begun tracking a file.

Create sample data on the SAME es-node1/Kibana/Logstash VM

```
sudo install -d -m 0755 /var/tmp/elk-labs/logs
# From the extracted ELK_Lab_Files directory:
sudo cp sample-data/*.log /var/tmp/elk-labs/logs/
sudo filebeat test config
sudo filebeat test output
sudo systemctl enable --now filebeat
sudo systemctl status filebeat --no-pager
```

Observe the stdout starter pipeline

```
sudo journalctl -u logstash -n 100 --no-pager
sudo journalctl -u filebeat -n 60 --no-pager
```

Check for the Filebeat-delivered log message and lab_source: filebeat. If no event appears, confirm that port 5044 is listening, log files match the glob, the data can be read, and the Filebeat registry has not already consumed the records. Create a fresh app-run2.log for repeat tests.

2A.7 | Upgrade from starter to the original L3 four pipelines

After the receiver test, follow the L3.2 routing and reliable-ingestion exercise in the retained manual. The original L3 architecture is still applicable because all four Logstash pipelines run inside the same Logstash process.

Copy configuration files from updated ZIP on es-node1

```
cd ~/ELK_Lab_Files
sudo install -d -m 0750 /etc/logstash/lab
sudo cp logstash/{10,20,30,40}-*.conf /etc/logstash/lab/
sudo chown -R root:logstash /etc/logstash/lab
sudo chmod 0750 /etc/logstash/lab
sudo chmod 0640 /etc/logstash/lab/*.conf
sudo cp /etc/logstash/pipelines.yml \
  /etc/logstash/pipelines.yml.before-l3
sudo cp logstash/pipelines.yml /etc/logstash/pipelines.yml
```

Prevent two Beats inputs binding port 5044

```
# Keep the prior starter file for reference, not active:
sudo mv /etc/logstash/conf.d/filebeat.conf \
  /etc/logstash/conf.d/filebeat.conf.starter-disabled
# Confirm the 4 pipeline definitions:
sudo cat /etc/logstash/pipelines.yml
```

The updated pipeline pack uses lab_ingress -> lab_parse -> lab_indexer / lab_quarantine. The good and failed output pipelines use small persistent queues. Put the Logstash Elasticsearch credentials into the keystore as explained on the next page, then validate as the logstash user.

Validate and restart in a controlled order

```
sudo -u logstash /usr/share/logstash/bin/logstash \
  --path.settings /etc/logstash --config.test_and_exit
sudo systemctl restart logstash
sudo systemctl status logstash --no-pager
curl -s http://127.0.0.1:9600/_node/pipelines?pretty
```

> FIELD NOTE A test that prints Configuration OK validates syntax but does not check all Grok patterns or confirm connectivity/authorization to Elasticsearch. Use Kibana count queries to complete the end-to-end test.

2A.8 | Secure Logstash -> remote Elasticsearch output

The Elasticsearch service was inactive on es-node1; the real Elasticsearch nodes are remote. Therefore Logstash must not target localhost:9200 for this deployment. Set ES_URL to the HTTPS endpoint of a running cluster node that passes certificate validation.

Verify the TLS endpoint and obtain its public HTTP CA certificate

```
# ON AN ACTUAL ELASTICSEARCH NODE (node01/node02/node03):
sudo ls -l /etc/elasticsearch/certs/http_ca.crt
# Copy only the PUBLIC CA, not node private keys.
sudo cp /etc/elasticsearch/certs/http_ca.crt /tmp/lab-http-ca.crt
sudo chmod 0644 /tmp/lab-http-ca.crt
# Transfer over approved scp or secure VM-shared file transfer.
# ON es-node1 (Kibana + Logstash):
sudo install -d -m 0755 /etc/logstash/certs
sudo install -m 0644 ~/lab-http-ca.crt \
  /etc/logstash/certs/http_ca.crt
```

Transfer the exported CA from one authorized Elasticsearch node to es-node1 using scp USERNAME@192.168.122.227:/tmp/lab-http-ca.crt ~/lab-http-ca.crt, changing USERNAME to the actual remote SSH account. Copy it into /etc/logstash/certs as above. The IPs .227, .246 and .102 are from your screenshot. Select a URL with a hostname or IP present in the node certificate SAN.

Logstash keystore and writer identity

```
# Run on es-node1; create only if keystore does not exist.
sudo /usr/share/logstash/bin/logstash-keystore \
  --path.settings /etc/logstash add ES_URL
sudo /usr/share/logstash/bin/logstash-keystore \
  --path.settings /etc/logstash add ES_USER
sudo /usr/share/logstash/bin/logstash-keystore \
  --path.settings /etc/logstash add ES_PASSWORD
sudo /usr/share/logstash/bin/logstash-keystore \
  --path.settings /etc/logstash add ES_CA_CERT
sudo chown logstash:logstash /etc/logstash/logstash.keystore
sudo chmod 0640 /etc/logstash/logstash.keystore
```

Set values at prompts: ES_URL = https://<verified-ES-host>:9200; ES_USER = scoped training writer; ES_PASSWORD = private secret; ES_CA_CERT = /etc/logstash/certs/http_ca.crt. Create the training-l3-* index writer role as in the retained manual. Do not disable TLS verification or paste passwords in screenshots.

2A.9 | Verification gates, failures and safe load testing

Run and record each gate before starting L3.4

Gate	Command / evidence	Pass / action
1. Logstash syntax	sudo -u logstash ... --config.test_and_exit	Configuration OK, no permission-denied errors
2. Logstash daemon	systemctl status logstash	active (running)
3. Beats receiver	ss -lntp | grep 5044	listening on loopback port 5044
4. Filebeat config	sudo filebeat test config	Config OK
5. Filebeat output	sudo filebeat test output	Connects to 127.0.0.1:5044
6. Source records	journalctl -u logstash	Event printed for stdout starter
7. Full L3 routing	GET training-l3-good/_count	4 good / 1 failed on first clean test

Kibana Dev Tools: only after the full L3 output is configured

```
GET training-l3-good/_count
GET training-l3-failed/_count
GET training-l3-good/_search?size=2
GET training-l3-failed/_search?size=2
```

If indices do not exist, inspect Logstash output logs, the keystore, Elasticsearch endpoint and writer role. The initial stdout starter pipeline intentionally does not create indices.

Safe L3.4 scope

With 3.1 GiB available RAM and no swap observed, limit load to small batches at first. Do not stop node01, node02 or node03 to induce backpressure on the live 3-node classroom cluster. Use a dedicated disposable test endpoint, preapproved isolated output disruption, or omit the outage exercise and document why.

> FIELD NOTE Trainer record: verified facts - Kibana and cluster visible; Filebeat installed; Logstash installed; non-root syntax check eventually showed Configuration OK; the log-file permission error was diagnosed. Start-up success, successful Filebeat delivery, and Elasticsearch indexing remain to be demonstrated.

2A.10 | Troubleshooting quick reference & lab evidence

Symptom	Cause / correction
Logstash cannot be run as superuser	Do not launch the Logstash binary with sudo alone. Use sudo -u logstash; systemctl service runs under configured account.
Unable to locate appender / plain.log permission denied	Check ownership of /var/log/logstash; restore logstash:logstash ownership for the log folder; rerun non-root validation.
Config OK but no 5044 listener	Config syntax valid; start service and inspect journalctl. Confirm 5044 not occupied by a second pipeline.
Filebeat output.elasticsearch localhost:9200	Disable it and enable output.logstash to 127.0.0.1:5044 on the same VM.
Filebeat config OK but no logs in Logstash	Check source glob, filesystem permissions, filestream registry/fingerprint, Logstash listener and logs.
Logstash -> Elasticsearch failure	ES_URL must be a remote HTTPS cluster node; confirm CA/SAN/credentials and training-l3-* permissions.
Only one service installed on a VM?	Do not assume the hostname describes the role; inspect systemctl states and Kibana _cat/nodes.

For the separate JSON/CSV/XML one-off Logstash exercise, create a writable alternate data directory first: sudo install -d -o logstash -g logstash -m 0750 /var/tmp/elk-labs/logstash-formats, then run the Logstash binary as sudo -u logstash. This replaces root execution in the retained example.

Evidence files to submit

- 01_hostname_services.png - es-node1 hostname + four service states
- 02_cluster_nodes.png - node01/node02/node03 and elected master
- 03_filebeat_version.png - Filebeat 9.5.5
- 04_logstash_config_ok.png - non-root Configuration OK
- 05_logstash_permission_resolution.png - clean permission/ownership test
- 06_logstash_running_5044.png - systemctl + ss listener
- 07_filebeat_output_test.png - Filebeat connects to local Logstash
- 08_l3_good_failed_counts.png - Elasticsearch good/failed indices

Subsequent activities retained from original manual

L3: multiline, Grok/Mutate/Date, routing, persistent queues, DLQ, JSON/CSV/XML and load; L4: ingest simulation, templates, mappings, data streams and ILM; Q2: match/term/range/bool queries. These are not omitted by the field corrections.

Official validation: https://www.elastic.co/docs/reference/beats/filebeat/logstash-output ; https://www.elastic.co/docs/release-notes/logstash/breaking-changes ; https://www.elastic.co/docs/reference/logstash/config-setting-files

2B | Environment discovery, staging and secure configuration

Step 1 - verify versions and health

From your Ubuntu terminal, run these where installed; use Kibana Dev Tools for Elasticsearch API requests. Record the returned major versions in the lab evidence.

```
# Run on each relevant VM, after Section 2A install
sudo systemctl status logstash --no-pager   # Logstash VM
filebeat version                            # es-node1 VM
sudo ss -lntp | grep -E ':(5044|5601|9200|9600)'
df -h /var/lib/logstash /var/tmp
```

```
GET /
GET _cluster/health
GET _nodes?filter_path=nodes.*.version
GET _license
```

Expected: reachable cluster, coherent node versions and enough disk. A single-node yellow cluster may simply have unassigned replicas; the L4 lab sets zero replicas only for its own indices.

Step 2 - create test paths and stage log data

```
# ES-NODE1; sample logs were staged in Section 2A
ls -lah /var/tmp/elk-labs/logs
# LOGSTASH VM; from ~/ELK_Lab_Files
sudo install -d -m 0750 /etc/logstash/lab
sudo cp logstash/{10,20,30,40}-*.conf /etc/logstash/lab/
sudo chown -R root:logstash /etc/logstash/lab
sudo chmod 0750 /etc/logstash/lab
sudo chmod 0640 /etc/logstash/lab/*.conf
```

The relative paths above refer to the unzipped companion pack. Substitute the actual directory where you extracted the ZIP. Copying to /etc/logstash requires sudo.

Step 3 - configure TLS and secret substitution

On the dedicated Logstash VM, the Elasticsearch HTTP CA has already been copied and the keystore populated in Section 2A. Do not repeat the create command if the keystore exists. Confirm that the trusted CA belongs to the Elasticsearch endpoint specified in ES_URL.

```
sudo install -d -m 0755 /etc/logstash/certs
# Example only when ES and Logstash share a VM:
sudo cp /etc/elasticsearch/certs/http_ca.crt \
  /etc/logstash/certs/http_ca.crt
sudo chmod 0644 /etc/logstash/certs/http_ca.crt
sudo /usr/share/logstash/bin/logstash-keystore \
  --path.settings /etc/logstash create
sudo /usr/share/logstash/bin/logstash-keystore \
  --path.settings /etc/logstash add ES_URL ES_USER ES_PASSWORD ES_CA_CERT
```

When prompted, enter values separately: ES_URL=https://<es-host>:9200, ES_USER=a lab account, ES_PASSWORD=its secret, ES_CA_CERT=/etc/logstash/certs/http_ca.crt. Restrict keystore file access so

REVISED SETUP: Elasticsearch CA must come from a real remote ES node; not localhost

the Logstash service account can read it. Prefer a scoped training user rather than a superuser.

TRAINER NOTE If an existing keystore exists, do not overwrite it; add the four keys. The Logstash configuration uses ${ES_*} interpolation. Never commit secrets to the companion ZIP or screenshots.

3 | L3.1 - Filebeat selection and multiline boundaries

Objective: Select app-*.log files while excluding app-debug.log; correctly turn a Java exception and its stack trace into a single Filebeat event.

Step 1 - inspect sample log lines

File: sample-data/app-training.log

```
2026-11-02T09:00:00.123Z INFO [checkout] order_id=ORD-1001 user=asha duration_ms=23
  message="Checkout completed"
2026-11-02T09:00:01.456Z ERROR [checkout] order_id=ORD-1002 user=ravi duration_ms=412
  message="Payment gateway failed"
java.lang.IllegalStateException: gateway unavailable
    at com.example.Checkout.pay(Checkout.java:42)
    at com.example.Service.handle(Service.java:21)
2026-11-02T09:00:02.000Z WARN [catalog] order_id=ORD-1003 user=maya duration_ms=87
  message="Cache miss"
2026-11-02T09:00:03.000Z UNKNOWN_LINE_THAT_FAILS_GROK
2026-11-02T09:00:04.000Z INFO [checkout] order_id=ORD-1004 user=lee duration_ms=35
  message="Payment settled"
```

This data contains five application events: four valid messages (one with a 3-line stack trace) and one deliberately malformed header that still starts with a timestamp. A debug file and an unrelated misc.log file are present to verify file selection.

Step 2 - backup and set Filebeat configuration

```
sudo cp /etc/filebeat/filebeat.yml \
  /etc/filebeat/filebeat.yml.before-elk-labs
sudo cp filebeat/filebeat-lab.yml /etc/filebeat/filebeat.yml
sudo chown root:root /etc/filebeat/filebeat.yml
sudo chmod 0644 /etc/filebeat/filebeat.yml
```

File: filebeat/filebeat-lab.yml

REVISED SETUP: use output.logstash hosts 127.0.0.1:5044, not the fifth-VM IP

```
filebeat.inputs:
  - type: filestream
    id: training_app_logs
    enabled: true
    paths:
      - /var/tmp/elk-labs/logs/app-*.log
    exclude_files: ['app-debug\.log$']
    prospector.scanner.check_interval: 2s
    prospector.scanner.fingerprint.length: 64
    parsers:
      - multiline:
          type: pattern
          pattern: '^[0-9]{4}-[0-9]{2}-[0-9]{2}T'
          negate: true
          match: after
          timeout: 2s
          max_lines: 100
    fields:
```

```
      training: l3
output.logstash:
  hosts: ['127.0.0.1:5044    ]
logging.level: info
```

Interpretation: a line beginning with an ISO-like timestamp starts a new message. Non-matching lines are appended to the prior event (negate: true, match: after). Filebeat performs multiline aggregation before shipping to Logstash; this is safer than combining mixed Beats streams inside Logstash.

Step 3 - validate and start Filebeat after Logstash is ready

```
sudo filebeat test config -c /etc/filebeat/filebeat.yml -e
sudo filebeat test output -c /etc/filebeat/filebeat.yml -e
sudo systemctl restart filebeat
sudo journalctl -u filebeat -n 35 --no-pager
```

If the test output fails because Logstash is not yet listening on 5044, finish Section 4 first, then repeat this step. Make a copy of the data before rerunning; Filebeat tracks offsets and may not resend unchanged lines.

Step 4 - validation questions

1. Run GET training-l3-good/_count and GET training-l3-failed/_count once Logstash is running; expect 4 good and 1 malformed event for a fresh first pass.

2. Search for order.id = ORD-1002 and inspect event.original; confirm the exception and both “at ...” lines are in the same event.

3. Confirm neither ORD-IGNORE nor ORD-IGNORE2 appears in the training indices. Record why glob paths and exclude_files behave differently.

4 | L3.2 - Build parsing and pipeline-to-pipeline routing

Objective: separate transport, transformation, good-event indexing and quarantine handling into independently observable Logstash pipelines.

Step 1 - architecture and routing

```
Filebeat --> lab_ingress [beats:5044]
                    | pipeline output: parse_bus
                    v
              lab_parse [Grok / Date / Mutate]
                    |                     |
                 success               parse failure
                    v                     v
             index_bus                quarantine_bus
                    |                     |
             lab_indexer           lab_quarantine
                [PQ]                    [PQ]
                    v                     v
          training-l3-good       training-l3-failed
```

Pipeline-to-pipeline buses are local to one Logstash process; they are not network endpoints. The downstream pipeline declares pipeline { address => ... }, while the sender uses pipeline { send_to => [...] }. Avoid cyclic dependencies.

Step 2 - dedicated pipelines.yml

File: logstash/pipelines.yml

```
# Copy these entries to a dedicated lab Logstash instance only.
- pipeline.id: lab_ingress
  path.config: /etc/logstash/lab/10-ingress.conf
  pipeline.workers: 1
- pipeline.id: lab_parse
  path.config: /etc/logstash/lab/20-parse.conf
  pipeline.workers: 2
- pipeline.id: lab_indexer
  path.config: /etc/logstash/lab/30-indexer.conf
  pipeline.workers: 2
  pipeline.batch.size: 125
  pipeline.batch.delay: 50
  queue.type: persisted
  queue.max_bytes: 64mb
  queue.page_capacity: 16mb
- pipeline.id: lab_quarantine
  path.config: /etc/logstash/lab/40-quarantine.conf
  pipeline.workers: 1
```

```
  queue.type: persisted
  queue.max_bytes: 64mb
  queue.page_capacity: 16mb
```

Copy this file to /etc/logstash/pipelines.yml only on a dedicated test instance, after backing up the original. The two output pipelines use persisted queues, each bounded to 64 MB. Queue sizing is per pipeline.

Step 3 - ingress and parser configuration

File: logstash/10-ingress.conf

```
input { beats { id => "lab_beats" port => 5044 } }
output { pipeline { send_to => ["parse_bus"] } }
```

File: logstash/20-parse.conf

```
input { pipeline { address => "parse_bus" } }
filter {
  grok {
    match => {
      "message" => '^%{TIMESTAMP_ISO8601:app_ts} %{LOGLEVEL:app_level} \[%{DATA:app_service}\]
        order_id=%{NOTSPACE:order_id} user=%{NOTSPACE:app_user} duration_ms=%{INT:duration_ms:int}
        message="%{DATA:app_message}"'
    }
    tag_on_failure => ["_lab_grok_failure"]
  }
  if "_lab_grok_failure" not in [tags] {
    date {
      match => ["app_ts", "ISO8601"]
      target => "@timestamp"
      tag_on_failure => ["_lab_date_failure"]
    }
    mutate {
      copy => { "message" => "[event][original]" }
      rename => {
        "app_level" => "[log][level]"
        "app_service" => "[service][name]"
        "app_user" => "[user][name]"
        "order_id" => "[order][id]"
        "duration_ms" => "[app][duration_ms]"
        "app_message" => "[app][summary]"
      }
      add_field => { "[labels][training_lab]" => "L3" }
    }
  }
  if "_lab_grok_failure" in [tags] or "_lab_date_failure" in [tags] {
    mutate {
      add_tag => ["training_parse_failed"]
      add_field => { "[failure][reason]" => "grok_or_date_failed" }
    }
  }
}
```

```
output {
  if "training_parse_failed" in [tags] {
    pipeline { send_to => ["quarantine_bus"] }
  } else {
    pipeline { send_to => ["index_bus"] }
  }
}
```

The parser extracts app_ts, level, service, order_id, user, duration and summary. Date assigns the application timestamp to @timestamp. Mutate creates structured fields and preserves the full multiline text in event.original. Any Grok/Date failure receives the training_parse_failed tag and is directed to quarantine; this is a deliberate application-level routing rule, not an automatic DLQ feature.

REVISED COMMAND: validate with sudo -u logstash (never sudo as root)

Step 4 - output pipelines

File: logstash/30-indexer.conf

```
input { pipeline { address => "index_bus" } }
output {
  elasticsearch {
    id => "lab_es_good"
    hosts => ["${ES_URL}"]
    user => "${ES_USER}"
    password => "${ES_PASSWORD}"
    ssl_certificate_authorities => ["${ES_CA_CERT}"]
    index => "training-l3-good"
    data_stream => false
    ilm_enabled => false
  }
}
```

File: logstash/40-quarantine.conf

```
input { pipeline { address => "quarantine_bus" } }
output {
  elasticsearch {
    id => "lab_es_failed"
    hosts => ["${ES_URL}"]
    user => "${ES_USER}"
    password => "${ES_PASSWORD}"
    ssl_certificate_authorities => ["${ES_CA_CERT}"]
    index => "training-l3-failed"
    data_stream => false
    ilm_enabled => false
  }
}
```

TRAINER NOTE The output plugin example uses ssl_certificate_authorities (recent Elastic versions). If an older installed plugin does not recognize it, check the installed plugin documentation and its equivalent CA parameter. Do not disable TLS verification.

Step 5 - enable Logstash monitoring and DLQ, then validate

File: logstash/logstash.yml additions

```
# Add only to the DEDICATED training Logstash /etc/logstash/logstash.yml.
api.http.host: "127.0.0.1"
api.http.port: 9600
dead_letter_queue.enable: true
# Persistent queues are configured per pipeline in pipelines.yml.
```

```
sudo cp /etc/logstash/pipelines.yml \
 /etc/logstash/pipelines.yml.before-elk-labs
sudo cp logstash/pipelines.yml /etc/logstash/pipelines.yml
# Add settings from logstash.yml-additions.txt into /etc/logstash/logstash.yml
sudo -u logstash /usr/share/logstash/bin/logstash --path.settings /etc/logstash \
  --config.test_and_exit
sudo systemctl restart logstash
sudo journalctl -u logstash -n 60 --no-pager
curl -s http://127.0.0.1:9600/_node/pipelines?pretty
```

If the validation command warns that pipelines.yml is ignored, check whether Logstash is running with -f or centralized management enabled. Confirm four named pipelines in the monitoring response before starting Filebeat.

Step 6 - inspect parsed, failed and multiline records

```
GET training-l3-good/_count
GET training-l3-failed/_count
GET training-l3-good/_search
{
  "size": 10,
  "query": { "term": { "order.id.keyword": "ORD-1002" } },
  "_source": ["@timestamp", "log", "service", "app", "event.original"]
}
GET training-l3-failed/_search
{ "query": { "term": { "tags.keyword": "training_parse_failed" } } }
```

Because L3 uses automatically inferred mappings, some dynamic keyword paths vary by Elastic version. If a term query returns zero unexpectedly, inspect GET training-l3-good/_mapping or search using a matching available exact field. In most recent stacks, dynamically mapped strings have a .keyword subfield.

5 | L3.3 - Grok, JSON, CSV, XML and error handling

Step 1 - Grok and Date exercise

1. On a copy of 20-parse.conf, change the expected literal order_id= to order_code= in the Grok pattern only; rerun configuration validation and watch how some records move into training-l3-failed.

2. Restore the proper pattern. Compare app_ts (raw text) with @timestamp (date) in good records. Explain how using ingestion time instead of event time changes range searches.

3. Change duration_ms to a nonnumeric token in one newly appended record. Observe that Grok with %{INT:...:int} fails and the record is quarantined.

TRAINER NOTE Use unique new log lines for retests rather than overwriting an already-harvested file: Filebeat maintains registry offsets and identities. For a clean repeat, create a new app-run2.log file.

Step 2 - one-off format parsing micro-lab

This separate lab demonstrates JSON, CSV and XML Logstash filters without modifying the main four pipelines. It generates three records internally, parses each into a parsed structure and prints them to stdout. The generator count is one, so no production data is involved.

File: logstash/90-format-mini-lab.conf

```
# Run ONLY as an independent one-off pipeline, not in pipelines.yml.
input {
  generator {
    count => 1
    lines => [
      'JSON|{"order_id":"ORD-9","amount":75,"status":"paid"}',
      'CSV|ORD-10,settled,23.50',
      'XML|<order><id>ORD-11</id><status>pending</status></order>'
    ]
  }
}
filter {
  dissect { mapping => { "message" => "%{format}|%{payload}" } }
  if [format] == "JSON" {
    json { source => "payload" target => "parsed" }
  } else if [format] == "CSV" {
    csv {
      source => "payload"
      target => "parsed"
      columns => ["order_id", "status", "amount"]
      convert => { "amount" => "float" }
    }
  } else if [format] == "XML" {
    xml { source => "payload" target => "parsed" store_xml => true }
  }
}
output { stdout { codec => rubydebug } }
```

REVISED COMMAND: run one-off Logstash process as logstash user, with writable paths

```
# Check that installed plugins include json/csv/xml/dissect:
sudo /usr/share/logstash/bin/logstash-plugin list | \
  grep -E '(logstash-filter-json|logstash-filter-csv|logstash-filter-xml|logstash-filter-dissect)'
# Run as a SEPARATE test process with a UNIQUE data directory:
sudo -u logstash /usr/share/logstash/bin/logstash \
  --path.settings /etc/logstash \
  --path.data /var/tmp/elk-labs/logstash-formats \
  -f /etc/logstash/lab/90-format-mini-lab.conf
```

Before running, copy the one-off file to /etc/logstash/lab/. In the stdout event, confirm that JSON becomes parsed.order_id / amount, CSV amount is numeric, and XML creates a parsed object. Stop with Ctrl+C after the output appears; this one-off example does not terminate automatically.

Step 3 - parsing failure vs DLQ vs PQ

CASE	DESTINATION	WHY
Malformed input syntax	training-l3-failed	Explicit tag-and-route rule in lab_parse.
Output unavailable / retryable failure	Persistent queue (if accepted)	Queues unacknowledged events pending output recovery; capacity bounded.
Eligible Elasticsearch bulk item rejection	Logstash dead-letter queue	Typically per-document 400/404 failures or conditional evaluation errors; DLQ must be enabled.

A Logstash DLQ is not a general catch-all for Grok failures or HTTP connection outages. Transport-level Elasticsearch output failures normally cause retries; they do not automatically become DLQ entries. A DLQ entry must be explicitly inspected, corrected and replayed using the dead_letter_queue input.

Optional - DLQ inspection and replay template

File: logstash/95-dlq-replay-example.conf

```
# OPTIONAL, run in its own process after an actual Elasticsearch bulk item 400/404.
# Replace path with your dedicated Logstash instance's path.data/dead_letter_queue.
input {
  dead_letter_queue {
    path => "/var/lib/logstash/dead_letter_queue"
    pipeline_id => "lab_indexer"
    commit_offsets => true
  }
}
filter {
  mutate { add_tag => ["dlq_replayed"] }
  # Add a case-specific field fix BEFORE reindexing. Do not replay unchanged.
}
output { stdout { codec => rubydebug { metadata => true } } }
```

An actual DLQ exercise requires deliberately producing an Elasticsearch non-retryable bulk item rejection (for example, a mapping conflict in a separate test index). Do not claim a DLQ pass merely because the malformed Grok test succeeded. Verify a file under path.data/dead_letter_queue/lab_indexer/, inspect the reason and apply a field correction before reindexing. The supplied replay template prints records only, so it is non-destructive.

SAFETY: do not stop the running three-node cluster; use a disposable endpoint

6 | L3.4 - Backpressure, persistent queues and load

Step 1 - capture baseline pipeline metrics

From the dedicated Logstash VM, capture snapshots before sending extra load. The HTTP metrics API listens on localhost:9600; no Elasticsearch login is required for this local endpoint when configured as in the lab.

```
curl -s http://127.0.0.1:9600/_node/stats/pipelines?pretty \
  > /var/tmp/elk-labs/baseline.json
curl -s http://127.0.0.1:9600/_node/stats/pipelines/lab_indexer?pretty
curl -s http://127.0.0.1:9600/_node/stats/pipelines/lab_ingress?pretty
```

Record events.in, events.out, events.duration_in_millis and queue statistics where exposed. Newer versions can expose flow rates such as input_throughput, output_throughput and queue_backpressure; field presence varies by version.

Step 2 - generate bounded, fresh workload

```
python3 scripts/make_load.py --count 2000 --pad-bytes 0 \
  --output /var/tmp/elk-labs/logs/app-load.log
# Wait for Filebeat's scan + multiline timeout, then record metrics:
curl -s http://127.0.0.1:9600/_node/stats/pipelines?pretty \
  > /var/tmp/elk-labs/after-load.json
```

Compute average throughput for a measured interval: EPS = (events.out_after - events.out_before) / elapsed_seconds. Repeat the test with 2,000-5,000 fresh events and identical test conditions after recovery. Do not assume wall-clock command execution time equals ingestion completion time.

Step 3 - controlled output interruption

1. On a dedicated test Elasticsearch endpoint used exclusively by this lab, temporarily stop that endpoint using sudo systemctl stop elasticsearch. Do not stop a shared cluster node or any cluster with production responsibilities.

2. On the Logstash VM, append a bounded load with python3 scripts/make_load.py --count 10000 --pad-bytes 4096 --output /var/tmp/elk-labs/logs/app-load.log. This can create roughly tens of MB of synthetic data; monitor free space.

3. Repeatedly capture /_node/stats/pipelines/lab_indexer?pretty. Observe persistent queue occupancy / events_count, output retries and, if capacity is reached, upstream backpressure. A blocked output may not fill a large queue with too few events.

4. Restart the dedicated test endpoint with sudo systemctl start elasticsearch. Wait for recovery; confirm queue occupancy decreases and successful output counts rise. Avoid terminating Logstash abruptly during the drill.

5. Use GET training-l3-good/_count plus pipeline event counters as a consistency check; account for initial baseline and any retries. Report whether the bottleneck was Filebeat, Logstash filters/CPU or Elasticsearch output.

TRAINER NOTE If you cannot safely stop Elasticsearch, skip the interruption and use a specially isolated test endpoint/approved network-block drill instead. Never run OS-wide firewall rules or stop production services just to demonstrate backpressure.

Step 4 - controlled tuning (one variable at a time)

RUN	WORKERS	BATCH.SIZE	BATCH.DELAY	EVIDENCE
A baseline	2	125	50 ms	EPS / CPU / queue
B	1	125	50 ms	Repeat same load
C	2	250	50 ms	Repeat same load
D	2	125	10 ms	Repeat same load

Change only lab_indexer settings in pipelines.yml, restart the dedicated Logstash instance and rerun with newly named files. Output throughput may be limited by Elasticsearch latency and disk I/O; increasing workers or batch size does not guarantee improvement. Record latency, CPU saturation, dropped/failed events and output backpressure alongside EPS.

7 | L3 extension - centrally managed pipelines (licensed)

Centralized Logstash Pipeline Management is a subscription feature. It is not available in every license tier. Verify your installed Elastic subscription in Stack Management > License Management before presenting the feature.

1. Show the Kibana management interface for Logstash pipelines, when exposed by the current license and user role.

2. Create a small demonstration pipeline under a unique ID in a non-production instance or demonstrate the editor in read-only mode.

3. On a separate dedicated Logstash instance, configure xpack.management.enabled: true, the Elasticsearch hosts and registered pipeline IDs, along with approved authentication. Restart that instance.

4. Edit the pipeline centrally, validate its managed deployment and capture configuration history/screenshots if the UI supports it.

5. Explain the consequence: once centralized management is enabled on that Logstash instance, local pipelines.yml and path.config are inactive. Do not enable it on the local-pipeline L3 instance.

```
# Example settings concept; NOT for the local L3 instance:
xpack.management.enabled: true
xpack.management.elasticsearch.hosts: ["https://es-lab:9200"]
xpack.management.pipeline.id: ["centrally_managed_demo"]
# Add compatible secure authentication and TLS settings.
# Requires feature entitlement and suitable privileges.
```

Decision exercise - Logstash vs Elasticsearch ingest nodes

CRITERION	LOGSTASH	INGEST PIPELINE
Sources	Beats, Kafka, files, databases and plugins	Documents already reaching Elasticsearch
Processing	Rich conditionals, Grok, Date, CSV, XML, external outputs	Lightweight processors, Grok, Set, Convert, Script
Buffering	Persistent queues and backpressure management	No built-in equivalent to Logstash disk PQ
Routing	Multiple pipelines / outputs	Index routing and processor logic
Operational cost	Extra nodes / JVM / plugin lifecycle	Adds work to ingest-capable Elasticsearch nodes

Selection rule: use Elasticsearch ingest pipelines for simple transforms close to indexing; use Logstash when heterogeneous input handling, routing, PQ/DLQ capabilities or more complex plugin transformations justify the added tier. Both can be used together.

8 | L4.1 - Ingest pipeline simulation

Objective: build a deterministic ingest pipeline, simulate it without writing documents, and verify fields before attaching it to a data stream.

Step 1 - create processor pipeline

```
PUT _ingest/pipeline/training-enrich
{
  "description": "Training - simple Elasticsearch ingest enrichment",
  "processors": [
    {
      "set": {
        "field": "labels.training_lab",
        "value": "L4"
      }
    },
    {
      "uppercase": {
        "field": "log.level",
        "ignore_missing": true
      }
    },
    {
      "set": {
```

```
        "field": "ingest_status",
        "value": "processed"
      }
    }
  ]
}
```

The set processor adds labels.training_lab; uppercase normalizes log.level; set adds ingest_status. For records missing log.level, uppercase skips that field instead of failing.

Step 2 - simulate without indexing

```
POST _ingest/pipeline/training-enrich/_simulate?verbose=true
{
  "docs": [
    {
      "_index": "training-events-demo",
      "_id": "s1",
      "_source": {
        "@timestamp": "2026-11-02T10:00:00Z",
        "message": "Payment timeout",
        "log": {
          "level": "error"
        },
        "service": {
          "name": "checkout"
        },
        "duration_ms": 520
      }
    },
```

```
    {
      "_index": "training-events-demo",
      "_id": "s2",
      "_source": {
        "@timestamp": "2026-11-02T10:01:00Z",
        "message": "Login success",
        "log": {
          "level": "info"
        },
        "service": {
          "name": "identity"
        },
        "duration_ms": 45
      }
    }
  ]
}
```

Inspect the docs[].doc._source result and per-processor status. Both lowercase log.level values should become uppercase, and each record should contain labels.training_lab = L4 and ingest_status = processed. This simulation alone does not create any index or store documents.

Step 3 - failure investigation

1. Temporarily change uppercase.field to a nonexistent nested field and remove ignore_missing. Simulate again; inspect the processor error.

2. Restore the pipeline before continuing. Save the successful simulation response as L4 evidence.

3. Describe how on_failure handlers could tag a bad input without silently dropping it.

9 | L4.2 - Static/dynamic mappings and composable templates

Templates must exist before a matching data stream or index is created. The explicit mappings define known field types; the dynamic mapping policy allows Elasticsearch to infer new fields for previously unseen keys.

Step 1 - create the tiny ILM policy

```
PUT _ilm/policy/training-events-ilm
{
  "policy": {
    "phases": {
      "hot": {
        "actions": {
          "rollover": {
            "max_docs": 3
          }
        }
      },
      "delete": {
        "min_age": "1d",
        "actions": {
          "delete": {}
        }
      }
    }
```

```
  }
}
```

The hot phase uses max_docs = 3 for quick demonstration; the delete action begins at min_age = 1d. The manual never waits a full day for deletion. This policy must never be attached to non-lab data.

Step 2 - create mapping component template

```
PUT _component_template/training-mappings-v1
{
  "template": {
    "mappings": {
      "dynamic": true,
      "properties": {
        "@timestamp": {
          "type": "date"
        },
        "message": {
          "type": "text"
        },
        "log": {
          "properties": {
            "level": {
              "type": "keyword"
            }
          }
```

```
        },
        "service": {
          "properties": {
            "name": {
              "type": "keyword"
            }
          }
        },
        "duration_ms": {
          "type": "integer"
        },
        "ingest_status": {
          "type": "keyword"
        }
      }
    }
  }
}
```

Fields explicitly defined here are static types. dynamic: true allows new fields such as environment to be added when actual data is indexed. For strict schemas, switch to dynamic: strict in a separate experimental index to observe rejection on unknown fields.

Step 3 - create settings component template

```
PUT _component_template/training-settings-v1
{
  "template": {
    "settings": {
      "index.number_of_shards": 1,
      "index.number_of_replicas": 0,
      "index.lifecycle.name": "training-events-ilm",
      "index.default_pipeline": "training-enrich"
    }
  }
}
```

This maps the ILM policy and default ingest pipeline onto each new backing index. A single-node lab sets number_of_replicas = 0; make an intentional resiliency choice in a multi-node real cluster.

Step 4 - compose the index template and simulate resolution

```
PUT _index_template/training-events-template
{
  "index_patterns": [
    "training-events-*"
  ],
  "priority": 300,
  "data_stream": {},
  "composed_of": [
    "training-mappings-v1",
    "training-settings-v1"
  ],
  "_meta": {
    "owner": "training",
    "purpose": "L4 data stream lab"
  }
}
```

```
POST _index_template/_simulate/training-events-template
GET _index_template/training-events-template
GET _component_template/training-mappings-v1
```

Confirm combined mappings, lifecycle settings and the data_stream declaration. The training-events-* pattern is deliberately disjoint from the built-in logs-*-* family; priority 300 resolves competing matches within this isolated namespace.

10 | L4.3 - Data streams, ILM and rollover

Step 1 - create the data stream

```
PUT _data_stream/training-events-demo
GET _data_stream/training-events-demo
GET _cat/indices/.ds-training-events-demo*?v&expand_wildcards=all
```

Expected: one hidden backing index with a generation suffix similar to 000001. The exact date in that backing-index name depends on creation time. Data streams require documents with an @timestamp field and create-only indexing semantics.

Step 2 - index deterministic sample records

In Kibana Console, run the following requests. A full six-record bulk payload is also included as elasticsearch/07-sample-documents.ndjson in the ZIP for curl-based ingestion. Use unique document IDs to avoid 409 conflicts on repeat.

```
PUT training-events-demo/_create/l4-1
{
  "@timestamp": "2026-11-02T10:00:00Z",
  "message": "payment timeout connecting to gateway",
  "log": { "level": "error" },
  "service": { "name": "checkout" },
  "duration_ms": 520,
  "environment": "training"
}
PUT training-events-demo/_create/l4-2
{
  "@timestamp": "2026-11-02T10:01:00Z",
  "message": "payment completed",
  "log": { "level": "info" },
  "service": { "name": "checkout" },
  "duration_ms": 90
}
PUT training-events-demo/_create/l4-3
```

```
{
  "@timestamp": "2026-11-02T10:02:00Z",
  "message": "cart timeout waiting on stock",
  "log": { "level": "warn" },
  "service": { "name": "checkout" },
  "duration_ms": 310
}
```

Step 3 - force manual rollover for a controlled demonstration

```
POST training-events-demo/_rollover
GET _data_stream/training-events-demo
GET _cat/indices/.ds-training-events-demo*?v&expand_wildcards=all
GET .ds-training-events-demo*/_ilm/explain?expand_wildcards=all
```

Expected: two backing indices, with the newer generation as the current write index. ILM may roll over on its own after a poll once max_docs has been reached; if that happened already, inspect generations before forcing a further rollover. A forced manual rollover is a validation shortcut, not proof that the periodic ILM scheduler executed the policy.

Step 4 - index the remaining three documents into the new write index

```
PUT training-events-demo/_create/l4-4
{"@timestamp":"2026-11-02T10:03:00Z",
 "message":"database connection timeout","log":{"level":"error"},
 "service":{"name":"billing"},"duration_ms":720}
PUT training-events-demo/_create/l4-5
{"@timestamp":"2026-11-02T10:04:00Z",
 "message":"login successful","log":{"level":"info"},
 "service":{"name":"identity"},"duration_ms":45}
PUT training-events-demo/_create/l4-6
{"@timestamp":"2026-11-02T10:05:00Z",
 "message":"payment rejected by issuer","log":{"level":"error"},
 "service":{"name":"checkout"},"duration_ms":430}
```

```
POST training-events-demo/_refresh
GET training-events-demo/_count
GET training-events-demo/_mapping
GET _data_stream/training-events-demo
GET .ds-training-events-demo*/_ilm/explain?expand_wildcards=all
```

Expected: exactly six documents if the lab starts from a clean stream. The environment field, absent from the explicit component mapping, should have a dynamic mapping in the backing index that first received l4-1. The ingest pipeline should have normalized every log.level to uppercase.

Step 5 - compare hot / warm / cold / frozen / delete

TIER / PHASE	ROLE	LAB IMPLEMENTATION
Hot	Active indexing and frequent queries	Used: rollover policy; write index remains hot.
Warm	Less frequent updates/reads, less costly storage	Concept only; no dedicated warm nodes.
Cold	Infrequently queried historical data	Concept only; tier allocation requires suitable nodes.
Frozen	Very infrequent search, often searchable snapshots	Concept only; architecture and licensing vary.
Delete	Remove expired backing indices per policy	Policy defined (1d), not waited for.

TRAINER NOTE A one-node lab does not simulate physical tier movement just because an ILM phase is named warm or cold. Check data-node roles, allocation requirements, snapshot repositories and licensed features before demonstrating real tier transitions.

11 | Q2 - Query DSL comparison and validation

Objective: identify ERROR records in a time/duration window while avoiding unnecessary scoring for exact filters. Run these in Kibana Dev Tools after all six L4 records are indexed.

Q2a - analyzed match query

```
GET training-events-demo/_search
{
  "size": 20,
  "query": { "match": { "message": "payment" } },
  "_source": ["message", "log.level", "service.name"]
}
```

Expected IDs: l4-1, l4-2 and l4-6. Why: match analyzes text in the message text field. Scores may differ based on relevance.

Q2b - term query on an exact keyword

```
GET training-events-demo/_search
{
  "size": 20,
  "query": { "term": { "log.level": "ERROR" } },
  "_source": ["message", "log.level"]
}
```

Expected IDs: l4-1, l4-4 and l4-6. Why: log.level is explicitly mapped as keyword and the ingest processor uppercase-normalized the stored value.

Q2c - bool.filter with keyword, numeric range

```
GET training-events-demo/_search
{
  "size": 20,
  "query": {
    "bool": {
      "filter": [
        { "term": { "log.level": "ERROR" } },
        { "term": { "service.name": "checkout" } },
        { "range": { "duration_ms": { "gte": 400, "lte": 600 } } }
      ]
    }
  },
  "_source": ["message", "duration_ms"]
}
```

Expected IDs: l4-1 and l4-6. Why: exact conditions run in filter context; they do not contribute to relevance score, which is unnecessary for operational filtering.

Q2d - selective scoring plus filter

```
GET training-events-demo/_search
{
  "size": 20,
  "query": { "bool": {
    "must": [ { "match": { "message": "payment" } } ],
    "filter": [ { "term": { "service.name": "checkout" } } ]
  } }
}
```

Expected IDs: l4-1, l4-2 and l4-6. The must clause scores message relevance; the service filter narrows results without affecting that score. Compare _score with Q2c.

Q2 written questions

1. Why is term on a text field usually the wrong choice for exact full-string matching?
2. Why would term log.level: error return no results when the indexed value is ERROR?
3. What is the operational benefit of moving environment, level and time restrictions into bool.filter?
4. Modify Q2c to use an @timestamp range from 10:00:00Z to 10:06:00Z and preserve expected IDs.

12 | Measurements, evidence and scoring rubric

Evidence checklist - screenshots or saved JSON results

- [E1] Elastic Stack version and cluster health snapshot; lab host / node list.
- [E2] Filebeat input configuration showing glob, exclude_files and multiline rules.
- [E3] Example event.original containing the complete Java stack trace as one event.
- [E4] Grok, Date, Mutate and four pipeline .conf files, plus pipelines.yml.
- [E5] One quarantine record with training_parse_failed and its original malformed header.
- [E6] Before/during/after Logstash pipeline and PQ metrics for the interruption.
- [E7] EPS calculation, workers/batch tuning comparison and a written bottleneck diagnosis.
- [E8] Ingest simulation result showing fields before/after processors.
- [E9] Component templates, index template, ILM policy and template simulation result.
- [E10] Data stream metadata, two backing indices and ILM explain snapshot.
- [E11] Dynamic field mapping for environment alongside explicit keyword/text/numeric mapping.
- [E12] Four query bodies, response IDs, hit totals and a brief explanation of scoring.

Acceptance rubric (100 points)

AREA	POINTS	CRITERIA
L3 selection / multiline	15	Correct file inclusion/exclusion and 1 combined stack trace.
L3 parsing / routing	20	4 parsed records, 1 quarantined record, valid configs.
L3 resilience / performance	20	Measured queue behavior + EPS and bottleneck analysis.
L4 ingest / mappings	15	Successful simulation, component/index templates, dynamic field.
L4 data stream / ILM	15	6 documents, 2 generations and lifecycle status evidence.
Q2 Query DSL	15	Four validated queries with correct expected record IDs.

Passing recommendation: at least 75/100, with no security violation or unapproved disruption. For central management and DLQ extensions, award separate bonus marks; both may depend on licensing or access to a deliberately controlled failure environment.

Measurement worksheet

RUN	IN BEFORE	OUT AFTER	ELAPSED SEC	EPS	QUEUE / BOTTLENECK
Baseline					
1 worker					
Batch 250					
Output blocked					
Recovered					

13 | Troubleshooting and safe reset

SYMPTOM	INVESTIGATION / ACTION
No events in Elasticsearch	Confirm port 5044, Filebeat test output, Logstash four pipelines, TLS CA and credentials.
Too few events	Check Filebeat registry offsets; create a new app-* file for replays, and wait for multiline timeout.
Stack trace split	Check filestream parsers and timestamp-start regex; avoid Logstash multiline codec for multiple Beats streams.
_grokparsefailure	Compare exact log format with anchored Grok pattern; inspect message and multiline boundaries.
Config syntax error	Use Logstash --config.test_and_exit; check quoted braces, file ownership and plugin installation.
Indexing 403	Assign test roles for indices, pipelines, templates, ILM and stream management; do not disable security.
Template ignored	Template must precede data stream creation; inspect index_patterns and priority.
Rollover not observed	Check nonempty backing index, ILM poll interval, policy status and manual rollover response.
DLQ directory empty	A Grok parse failure and an unreachable Elasticsearch node do not, by themselves, generate DLQ entries.
Queue never fills	The load may be below configured capacity; inspect events_count and queue occupancy, not assumptions.
Logstash settings ignored	Check -f launch parameter and xpack.management.enabled; central mgmt overrides local pipeline configs.

Cleanup - only when evidence has been exported

```
# Kibana Dev Tools -- deleting a data stream is destructive:
DELETE _data_stream/training-events-demo
DELETE _index_template/training-events-template
DELETE _component_template/training-mappings-v1
DELETE _component_template/training-settings-v1
DELETE _ingest/pipeline/training-enrich
DELETE _ilm/policy/training-events-ilm
DELETE training-l3-good
DELETE training-l3-failed
```

Restore the backed-up Filebeat and Logstash configuration, restart only the dedicated lab services, and remove /var/tmp/elk-labs when no longer needed. Do not delete /var/lib/logstash queues before stopping Logstash and confirming all needed data was delivered. A DLQ requires separate safe retention and disposition.

Trainer sign-off

Lab participant: _____ Date: _____ Score: _____ / 100

Observed bottleneck: _____ Recovery demonstrated: [ ] Yes [ ] No

14 | Official reference material

Installation links: https://www.elastic.co/docs/reference/logstash/installing-logstash and https://www.elastic.co/docs/reference/beats/filebeat/setup-repositories . Filebeat filestream fingerprint guidance: https://www.elastic.co/docs/reference/beats/filebeat/filebeat-input-filestream .

Filebeat filestream multiline: https://www.elastic.co/docs/reference/beats/filebeat/multiline-examples

Logstash pipeline-to-pipeline: https://www.elastic.co/docs/reference/logstash/pipeline-to-pipeline

Persistent queues: https://www.elastic.co/docs/reference/logstash/persistent-queues

Dead-letter queues: https://www.elastic.co/docs/reference/logstash/dead-letter-queues

Logstash performance: https://www.elastic.co/docs/reference/logstash/tuning-logstash

Logstash centralized management: https://www.elastic.co/docs/reference/logstash/logstash-centralized-pipeline-management

Composable templates: https://www.elastic.co/docs/manage-data/data-store/templates

ILM create policy: https://www.elastic.co/docs/manage-data/lifecycle/index-lifecycle-management/configure-lifecycle-policy

Ingest pipeline simulate API: https://www.elastic.co/docs/api/doc/elasticsearch/operation/operation-ingest-simulate

Query/filter context: https://www.elastic.co/docs/reference/query-languages/query-dsl/query-filter-context

Exact term query: https://www.elastic.co/docs/reference/query-languages/query-dsl/query-dsl-term-query

References checked 02 November 2026. Endpoint and plugin behavior may vary by deployed Elastic Stack version. Validate features against the installed version and subscription before training.